



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Новосибирский государственный технический университет»



**НГТУ
НЭТИ**

**Факультет автоматики
и вычислительной техники**

Кафедра вычислительной техники
Лабораторная работа № 2
по дисциплине «Технология разработки программного обеспечения»

Java. Разработка класса для представления структуры данных

Студент Чумаков Илья

Группа АТМ-25

Преподаватель Токарев Вадим Геннадьевич

Новосибирск, 2025

1 Задание

Портировать одну из структур данных, приведенных в cprog.

Набор операций для структуры данных:

- добавление в конец, получение, вставка и удаление по логическому номеру (индексу). Логический номер – порядковый номер хранимого элемента в соответствии с последовательностью обхода структуры данных (перебора элементов)
- для исходно упорядоченной (например, «двоичное дерево») вставка с сохранением порядка, получение и удаление по логическому номеру (индексу)
- Итератор forEach – для Java: интерфейс обратного вызова с методом void `forEach(T v)`, итератора получает ссылку на интерфейсный объект (анонимный класс), обходит структуру данных и вызывает метод для каждого выбранного элемента. В Scala – формальный параметр = функция.
- сортировка, для исходно упорядоченной – балансировка = создание экземпляра структуры данных с равными размерностями компонент (например, сбалансированное дерево, либо список списков со списками нижнего уровня равной длины) Элемент структуры данных хранит целое число.

Выбранная структура:

6. Хэш-таблица: вектор, содержащий заголовки списков

2 Реализация

а. Класс Node

```
private class Node {  
    int value;  
    Node next;  
  
    Node(int value) {  
        this.value = value;  
    }  
}
```

Хэш-таблица реализована как массив заголовков списков, где каждый узел (Node) представляет элемент списка в корзине. `value` хранит целое число, `next` – ссылку на следующий узел.

При создании нового узла инициализируется только `value`, `next` = null.

Этот класс является внутренним для `HashTable` и используется для построения связанных списков в каждой корзине.

б. Класс HashTable

```

public class HashTable {
    private class Node { ...

    private Node[] heads;
    private int bucketCount;
    private int size;

    public HashTable() {
        this(bucketCount: 10); // Количество корзин по умолчанию
    }

    public HashTable(int bucketCount) {
        this.bucketCount = bucketCount;
        this.heads = new Node[bucketCount];
        this.size = 0;
    }
}

```

MyHashTable содержит массив heads (Node[]) - заголовки списков для каждой корзины, bucketCount - количество корзин (по умолчанию 10), size - общее количество элементов. Через этот объект происходят все операции: создание, добавление, удаление и т.д. Конструктор позволяет задать количество корзин или использовать значение по умолчанию.

```

// Add to end: хешируем значение и добавляем в конец списка соответствующей корзины
public void addToEnd(int value) {
    int hash = hash(value);
    Node newNode = new Node(value);
    if (heads[hash] == null) {
        heads[hash] = newNode;
    } else {
        Node current = heads[hash];
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }
    size++;
}
}

```

Метод addToEnd(int value): Аналог добавления в конец, но с учётом хэширования: вычисляется хэш, элемент добавляется в конец списка соответствующей корзины. Увеличивает size.

```
// Получаем значение по позиции в порядке обхода всех элементов, игнорируя хеш
public int get(int index) {
    if (index < 0 || index >= size) {
        throw new IndexOutOfBoundsException(s: "Index out of bounds");
    }
    int count = 0;
    for (int i = 0; i < bucketCount; i++) {
        Node current = heads[i];
        while (current != null) {
            if (count == index) {
                return current.value;
            }
            count++;
            current = current.next;
        }
    }
    throw new RuntimeException(message: "Unexpected error");
}
```

Метод `get(int index)`: возвращает элемент по логическому индексу. Обходит все корзины и списки последовательно, подсчитывая элементы до достижения `index`. Если `index` вне диапазона - выбрасывает исключение.

```

// Вставляем значение по позиции в порядке обхода всех элементов, игнорируя хеш
public void insert(int value, int index) {
    if (index < 0 || index > size) {
        throw new IndexOutOfBoundsException(s: "Index out of bounds");
    }
    if (index == size) {
        addToEnd(value);
        return;
    }
    int count = 0;
    for (int i = 0; i < bucketCount; i++) {
        Node current = heads[i];
        Node prev = null;
        while (current != null) {
            if (count == index) {
                Node newNode = new Node(value);
                if (prev == null) {
                    newNode.next = heads[i];
                    heads[i] = newNode;
                } else {
                    newNode.next = current;
                    prev.next = newNode;
                }
                size++;
                return;
            }
            prev = current;
            current = current.next;
            count++;
        }
    }
}

```

Метод insert(int value, int index): Вставляет элемент по логическому индексу. Обходит структуру последовательно, находит позицию и вставляет новый узел в соответствующий список корзины. Если index == size, использует addToEnd. Увеличивает size.

```
// Удаление по позиции в порядке обхода всех элементов, игнорируя хеш
public void delete(int index) {
    if (index < 0 || index >= size) {
        throw new IndexOutOfBoundsException(s: "Index out of bounds");
    }
    int count = 0;
    for (int i = 0; i < bucketCount; i++) {
        Node current = heads[i];
        Node prev = null;
        while (current != null) {
            if (count == index) {
                if (prev == null) {
                    heads[i] = current.next;
                } else {
                    prev.next = current.next;
                }
                size--;
                return;
            }
            prev = current;
            current = current.next;
            count++;
        }
    }
    throw new RuntimeException(message: "Unexpected error");
}
```

Метод `delete(int index)`: Удаляет элемент по логическому индексу. Обходит структуру, находит узел и удаляет его из списка. Уменьшает `size`.

```
// Итератор forEach с обратным вызовом
public interface Callback {
    void toDo(int v);
}

public void forEach(Callback callback) {
    for (int i = 0; i < bucketCount; i++) {
        Node current = heads[i];
        while (current != null) {
            callback.toDo(current.value);
            current = current.next;
        }
    }
}
```

Метод `forEach(Callback callback)`: Реализация паттерна «итератор с обратным вызовом»:

- передаём интерфейс с методом `void toDo(int v)`;

- метод обходит все корзины и списки, применяя действие к каждому элементу.

```
// Сортировка всех элементов в каждой корзине и создание новой хеш-таблицы
public HashTable sort() {
    HashTable newTable = new HashTable(bucketCount);
    for (int i = 0; i < bucketCount; i++) {
        List<Integer> bucketList = new ArrayList<>();
        Node current = heads[i];
        while (current != null) {
            bucketList.add(current.value);
            current = current.next;
        }
        Collections.sort(bucketList);
        for (int v : bucketList) {
            newTable.addToEnd(v);
        }
    }
    return newTable;
}
```

Метод sort(): Создаёт новую хэш-таблицу: собирает элементы каждой корзины в список, сортирует его с помощью Collections.sort(), затем добавляет обратно в новую таблицу (с сохранением хэш-распределения).

```
// Хэш-функция
private int hash(int value) {
    int h = Integer.hashCode(value);
    return (h < 0 ? -h : h) % bucketCount;
}
```

Метод hash(int value): Вычисляет хэш-код: использует Integer.hashCode, делает положительным и берёт по модулю bucketCount.

```
// Получаем текущий размер
public int size() {
    return size;
}
```

Метод size(): Возвращает общее количество элементов.

3 Тестирование

В методе main() реализованы тесты всех разработанных функций хэш-таблицы.

```

public static void main(String[] args) {
    System.out.println(x: "=== Тестирование хэш-таблицы ===\n");

    // 1. Создание пустой хэш-таблицы с 5 бакетами
    HashTable table = new HashTable(bucketCount: 5);
    System.out.println(x: "1. Создана пустая хэш-таблица с 5 бакетами.");
    printTableState(table);
    System.out.println();

    // 2. Добавление элементов в конец
    System.out.println(x: "2. Добавление элементов в конец:");
    int[] valuesToAdd = {10, 25, 7, 42, 18, 33, 7};
    for (int v : valuesToAdd) {
        table.addToEnd(v);
        System.out.printf(format: "    addToEnd(%d) -> размер: %d\n", v, table.size());
    }
    printTableState(table);
    printBucketState(table);
    System.out.println();

    // 3. Получение элементов по индексу
    System.out.println(x: "3. Получение элементов по логическому индексу:");
    for (int i = 0; i < table.size(); i++) {
        System.out.printf(format: "    get(%d) = %d\n", i, table.get(i));
    }
    System.out.println();

    // 4. Вставка элементов по индексу
    System.out.println(x: "4. Вставка элементов по индексу:");
    table.insert(value: 999, index: 0); // в начало
    System.out.println(x: "    insert(999, 0) - вставка в начало");
    printTableState(table);

    table.insert(value: 555, index: 3); // в середину
    System.out.println(x: "    insert(555, 3) - вставка в середину");
    printTableState(table);

    table.insert(value: 777, table.size()); // в конец
    System.out.println(x: "    insert(777, size) - вставка в конец");
    printTableState(table);
    System.out.println();

    // 5. Удаление элементов по индексу
    System.out.println(x: "5. Удаление элементов по индексу:");
    table.delete(index: 0); // удаляем 999 из начала
    System.out.println(x: "    delete(0) - удаление первого элемента");
    printTableState(table);

    table.delete(table.size() - 1); // удаляем последний
    System.out.println(x: "    delete(size-1) - удаление последнего элемента");
    printTableState(table);

    table.delete(index: 2); // удаляем из середины
    System.out.println(x: "    delete(2) - удаление из середины");
    printTableState(table);
    System.out.println();
}

```



```

// 6. Итерация с использованием forEach
System.out.println(x: "6. Итерация по всем элементам (forEach):");
table.forEach(v -> System.out.print(v + " "));
System.out.println(x: "\n");

// 7. Сортировка
System.out.println(x: "7. Сортировка хэш-таблицы:");
System.out.println(x: "    До сортировки:");
table.forEach(v -> System.out.print(v + " "));
System.out.println();

HashTable sortedTable = table.sort();
System.out.println(x: "    После сортировки (в новой таблице):");
sortedTable.forEach(v -> System.out.print(v + " "));
System.out.println(x: "\n");
printTableState(sortedTable);
printBucketState(sortedTable);
System.out.println(x: "\n=== Тестирование завершено успешно ===");
}

```

Вывод метода main()

=== Тестирование хэш-таблицы ===

1. Создана пустая хэш-таблица с 5 бакетами.
Текущее состояние (логический порядок): []

2. Добавление элементов в конец:

addToEnd(10) -> размер: 1
addToEnd(25) -> размер: 2
addToEnd(7) -> размер: 3
addToEnd(42) -> размер: 4
addToEnd(18) -> размер: 5
addToEnd(33) -> размер: 6
addToEnd(7) -> размер: 7

Текущее состояние (логический порядок): 10 -> 25 -> 7 -> 42 -> 7 -> 18 -> 33 (размер: 7)

Текущее состояние (по корзинам):

Корзина 0: 10 -> 25

Корзина 1: (пусто)

Корзина 2: 7 -> 42 -> 7

Корзина 3: 18 -> 33

Корзина 4: (пусто)

3. Получение элементов по логическому индексу:

get(0) = 10

get(1) = 25

get(2) = 7

Текущее состояние (логический порядок): 10 -> 25 -> 555 -> 7 -> 42 -> 7 -> 777 -> 18 (размер: 8)

delete(2) ? удаление из середины

Текущее состояние (логический порядок): 10 -> 25 -> 7 -> 42 -> 7 -> 777 -> 18 (размер: 7)

6. Итерация по всем элементам (forEach):

10 25 7 42 7 777 18

7. Сортировка хэш-таблицы:

До сортировки:

10 25 7 42 7 777 18

После сортировки (в новой таблице):

10 25 7 7 42 777 18

Текущее состояние (логический порядок): 10 -> 25 -> 7 -> 7 -> 42 -> 777 -> 18 (размер: 7)

Текущее состояние (по корзинам):

Корзина 0: 10 -> 25

Корзина 1: (пусто)

Корзина 2: 7 -> 7 -> 42 -> 777

Корзина 3: 18

Корзина 4: (пусто)

=== Тестирование завершено успешно ===